

A Memory-Efficient LLM Accelerator with Q-K Correlation Prediction using Cluster-Based Associative Array for Selective KV Accessing

Zikang Zhou, Kaiqi Chen, Xuyang Duan, Jun Han*

State Key Laboratory of Integrated Chips and Systems, Fudan University, Shanghai, China

Email: *junhan@fudan.edu.cn

Abstract—Attention-based LLMs excel in text generation but face redundant computations in autoregressive token generation. While KV cache mitigates this, it introduces increased memory access overhead as sequences grow. We propose Sella, a hardware-software co-design using cluster-based associative arrays to predict Q-K correlations, enabling selective KV cache access and reducing memory access without retraining. Sella includes a specialized accelerator featuring a prediction engine to improve performance and energy efficiency. Experiments show Sella achieves 2.1 $\times$, 93.8 $\times$, 31.4 $\times$, and 53.5 $\times$ speedup over SpAtten, Sanger, TITAN RTX GPU, and Xeon CPU, respectively, reducing off-chip memory access by up to 66% with negligible accuracy loss.

Index Terms—Large Language Models, KV Cache, Accelerator

I. INTRODUCTION

Large language models (LLMs) represent a significant technological breakthrough in the new era of artificial intelligence (AI) development. Models such as GPT-4 [1], Llama [2], and OPT [3] have become outstanding examples of transformer architecture in the field of natural language processing (NLP). LLMs have enabled numerous innovative applications, including virtual assistants, text generation, and intelligent search.

The prevalent LLMs are built on stacked decoder architectures and characterized by autoregressive inference shown in Fig.1(a) [4]. However, autoregressive inference introduces a problem: during each attention calculation, the keys and values (KVs) of previous tokens must be recomputed to retrieve contextual information, resulting in quadratic computational complexity relative to the sequence length.

KV cache is proposed to eliminate KVs' recomputation by caching all previous KVs. However, it introduces memory access proportional to the sequence length. Fig.1(b) shows that as token generation progresses, access to the KV cache gradually increases, eventually surpassing that of weights and becoming dominant. The *baseline* in Fig.1(c) illustrates that the bandwidth demand for accessing the KV cache also gradually increases to ensure inference latency. Although HBM (high-bandwidth memory) is available, the KV cache

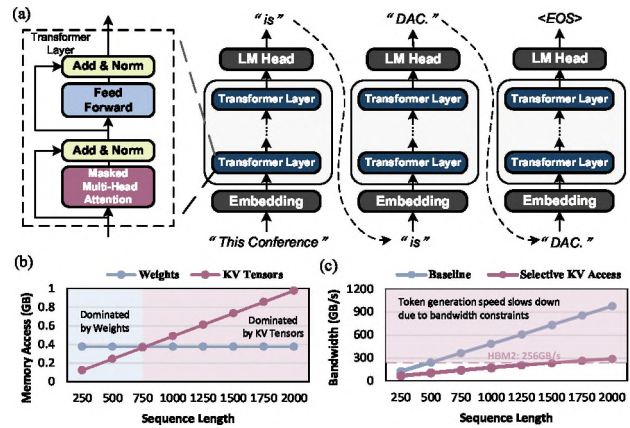


Fig. 1. Figure (a) shows the inference process of LLMs. Figures (b) and (c) show how memory access for Weights and KVs, as well as the bandwidth requirement for accessing the KV cache, change as the sequence length increases during inference for a single-layer decoder in OPT-6.7b with a batch size of 32. In Figure (c), the baseline represents the original bandwidth requirement, while selective KV access indicates the bandwidth requirement after applying the proposed technique.

bandwidth demand will eventually exceed what HBM can provide, becoming unacceptable and leading to a decline in token generation speed in practical applications.

A straightforward idea to reduce KV cache memory access is to load only the historical KVs strongly correlated with the current query for computation to alleviate the performance bottleneck under long context conditions. This is viable because prior research has demonstrated that not all preceding tokens contribute to generating the current token [5]–[10]. However, when these methods are applied to LLMs with KV cache, they face the following issues: high prediction overhead, inability to identify important KVs during token generation, and expensive retraining.

To address these issues, in this work, we propose a hardware-software co-designed solution for efficient KV accessing during token generation for LLMs, called Sella. Sella can reduce memory access and bandwidth requirements, as shown by the *selective KV access* in Fig.1(c). Note that although this paper primarily focuses on memory access, this method also reduces the attention computation. The contribu-

*Corresponding author.

This work was supported by the National Natural Science Foundation of China under Grant 61934002 and 62234008.

tions of this work are summarized as follows:

- We introduce a low-overhead, high-accuracy mechanism for predicting Q-K correlations using cluster-based associative arrays to selectively access the KV cache. This approach maintains tables that record correlation scores for K and approximated Q and determines important KVs dynamically during token generation through table lookups. Weights used to calculate the correlation scores are calibrated offline using clustering. This method ensures accuracy without costly retraining.
- We propose a high-performance, energy-efficient specialized LLM accelerator for text generation. This accelerator consists of a prediction engine and a computation engine. The prediction engine looks up and updates the associative array to predict significant KVs. The computation engine performs self-attention and is fully pipelined, achieving high throughput.
- We evaluated Sella across a wide range of LLMs and tasks. Experiments demonstrate that Sella significantly reduces memory access and inference time without significant accuracy loss, and decreases energy consumption.

II. BACKGROUND AND MOTIVATIONS

A. Large Language Models

Autoregressive inference. Autoregressive inference means generating each token sequentially, using each newly generated token with previously generated tokens as input to predict the next token. This process repeats until it reaches the model's max length or encounters an end-of-sequence token (EOS). It guarantees that each newly produced token relies on the existing context.

Attention and KV cache. The attention mechanism is the key component of LLMs, it computes the dependencies of each token on other tokens in the input sequence. It involves three tensors: Q (Query), K (Key), and V (Value). First, the Q and K tensors undergo an inner product, scaled by the square root of hidden dimension, and passed through a *softmax* to get the attention weights. Next, the attention weight is multiplied by tensor V to complete the attention computation [11].

Due to the autoregressive inference of LLMs, it is necessary to recompute all previous KVs when generating tokens for attention calculation. KV cache avoids recomputation by caching the previously computed K and V vectors. The computation flow is as follows: during the generation of each new token x_t , the corresponding k_t and v_t are calculated and stored in K_{cache} and V_{cache} , where K_{cache} is $\{k_1, \dots, k_t\}$ and V_{cache} is $\{v_1, \dots, v_t\}$. When generating a new token, the current token's q_{t+1} , k_{t+1} , and v_{t+1} are computed, and the attention computation is performed using the K_{all} and V_{all} , where $K_{all} = \{K_{cache}, k_{t+1}\}$, $V_{all} = \{V_{cache}, v_{t+1}\}$.

B. Motivations

Attention weights are sparse, and skipping KVs with weak dependencies has little impact on accuracy, as shown in Table I. However, predicting the important KVs before the actual attention computation is a challenge. Many previous works try

TABLE I
THE ACCURACY AND SPARSITY ACHIEVED BY PRUNING KVs ASSOCIATED WITH ATTENTION WEIGHTS UNDER 0.001

Network	Dataset	Dense Accuracy	Skipping Accuracy	KV Skipping Rate
Llama2-7b	COPA-16shots	0.920	0.920	0.781
	PIQA-4shots	0.781	0.786	0.769
Opt-6.7b	COPA-16shots	0.810	0.810	0.625
	PIQA-4shots	0.768	0.762	0.616

to predict the sparsity of attention weights to reduce attention computation [5]–[9]. However, these methods have the following limitations when applied to LLMs with KV cache. A³ [5] only reduces computational complexity but still requires all K vectors to participate in the calculation, thus cannot reduce memory access complexity. SpAtten [6] requires storing the entire attention weights for KV pruning decisions. It is only able to prune prompt tokens and cannot dynamically decide important KVs during the token generation. The predictions of Sanger [7] and DOTA [9] still require the inner product between each simplified Q and each simplified K to determine important KVs, leading to significant computational and memory access overhead during prediction. Moreover, both SpAtten [6] and DOTA [9] require retraining the networks, which is very expensive and not scalable for LLMs.

We expect to find a method to predict the important KVs before accessing the KV cache, with low overhead, dynamic prediction while token generation, and no need for costly retraining. We are inspired by clustering approximation. By using cluster centroids to approximate nearby vectors, calculating the inner product for every Q and K can be avoided. The next section will introduce the implementation using clustering approximation.

III. CLUSTER-BASED ASSOCIATIVE ARRAY FOR SELECTIVE KV ACCESSING

A. Clustering Approximation for Queries

Fig. 2 shows the process of clustering approximation. There are three query vectors: Q_1 , Q_2 , and Q_3 , and two cluster centroid vectors, H_1 and H_2 (both are unit vectors). Among them, Q_1 and Q_2 are close to H_1 (γ_{11} is small), while Q_3 is close to H_2 (γ_{32} is small). When performing dot product, the cosine values of the angle between other K vectors and Q_1 or Q_2 can be approximated as the cosine values of other K vectors and H_1 ; similarly, the cosine values between other K vectors and Q_3 can be approximated as the cosine values of K vectors and H_2 . During inference, the projection results of K vectors can be reused for different Q vectors to reduce computational costs (e.g., Q_1 and Q_2 in the figure reuse the $h_1(K_n)$ results), thus reducing the computation and relevant memory access.

B. Cluster-Based Associative Array

Based on clustering approximation, we implemented the associative array structure to reduce KV memory access during the LLM token generation process and accelerate inference.

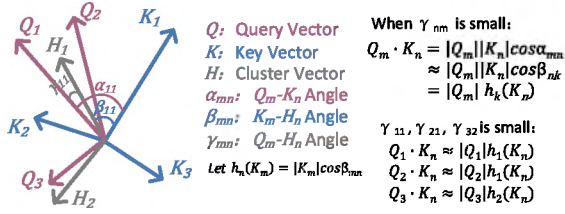


Fig. 2. The example of clustering approximation for KV dot-product.

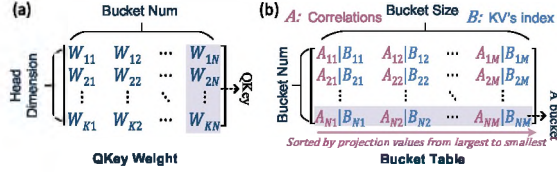


Fig. 3. The QKey weight and bucket table structure.

The associative array's keys (denoted as **QKeys**) are a set of centroids obtained by offline clustering of Q vectors. The associative array's values are finite-sized lists (denoted as **buckets**), which contain the several largest inner product values between QKeys and K , along with the corresponding indexes of K . The matrix formed by QKeys is denoted as **QKey weight**, and the array composed of buckets is denoted as **bucket table**. During inference, the Q vector is compared with the QKeys to find the best-matching bucket, and the KVs referenced in the bucket are used for computation. Next, we will introduce the components of the cluster-based associative array and its overall workflow.

1) **Bucket Table Structure**: Fig. 3(b) illustrates the bucket table structure used in this method. Each attention head in each layer maintains a bucket table. An element in the table takes 4 bytes and consists of two parts: the first few large projection values of the cached K onto the corresponding QKey (predicted Q - K correlations), and the corresponding token index. Larger predicted Q - K correlations indicate that the corresponding KVs are more important for this QKey. The number of rows in the table is denoted as BN (Bucket Num), representing the number of QKeys. Each row in the table is arranged in descending order of correlations. The number of columns in the table is denoted as BS (Bucket Size), which indicates the number of recorded elements in each bucket. BS grows proportionally as sequence length increases, which is $BS = \alpha * sequence_length$. Although the bucket table needs to be read and updated during each inference, this overhead is negligible compared to the saved memory access.

2) **QKey Weight Calibration**: Fig. 3(a) shows the QKey weight structure. Unlike the bucket table, all attention heads in each layer share a single QKey weight. The QKey weight is calibrated by clustering the Q vectors offline using the K-Means algorithm instead of retraining. First, we randomly sample data from the dataset for inference and record the Q vectors generated at each layer during the inference. Next, we set BN and perform clustering on the Q vectors generated by

each layer, getting BN cluster centroids, thereby forming a QKey weight of size $BN * HD$.

3) **Accuracy Compensation and Exception Handling**: For a Q vector, once the best-matching bucket is found, a simple and effective method is retrieving all the KVs recorded in that bucket. However, two situations can lead to accuracy issues. 1. Even if the Q vector is close to the nearest QKey, due to the relatively small BS , it may not be able to capture all important KVs, leading to a drop in accuracy. 2. Due to the small BN , the Q may be far from the nearest QKey, making it difficult to obtain the KVs that are important for that Q vector, resulting in excessive error and a decrease in accuracy.

For problem 1, although increasing BS is effective, it would greatly increase memory access overhead. Instead, we use elements from other buckets to perform accuracy compensation. After calculating the cosine values between Q and all the QKeys, the bucket with the highest cosine value retrieves all the cached KVs recorded in the bucket. For the other buckets, the top- N elements based on their cosine values are selected in the subsequent attention calculation, providing accuracy compensation. The N is determined by Eq.1.

$$N = round(max(0, cos\gamma) \cdot BS) \quad (1)$$

For problem 2, we set a threshold δ . When the cosine value between Q and the nearest QKey is less than δ , it is considered an exception. In this case, the bucket table is considered unreliable for that Q . At this time, no selection of KVs is performed. Instead, all cached KVs are directly accessed for attention computation.

4) **Overall workflow**: Fig.4 illustrates the workflow using associative arrays for selective KV accessing.

Bucket Table Update. ❶ At the beginning, the bucket table is loaded from the DRAM. ❷ Meanwhile, the K vector is multiplied by the QKey weights to obtain the predicted Q - K correlations for each bucket. ❸ Next, the predicted Q - K correlations obtained in step ❷ are used to update the bucket table. During the update, the predicted correlations and token index are inserted into the corresponding position in the table based on the relative magnitude of the correlations, and the element with the smallest correlation in each bucket is removed. There are no insertions if the correlation is smaller than all existing values in the bucket. ❹ After the update is complete, the bucket table is stored back into the DRAM for use in the inference of the next token.

Estimation of significant KVs. ❺ computes the cosine values between Q and all QKeys, which is $\frac{Q \cdot QKey}{||Q||}$, obtaining a list of all cosine values for Q on QKeys. ❻ Next, use this list of cosine values to look up the bucket table obtained in step ❸. If the largest value in the list is greater than δ , all cached KVs referenced in the nearest bucket are used for attention calculation, and the KVs referenced in other buckets are used for accuracy compensation as described above. Otherwise, the previously mentioned exception handling is performed. ❼ After obtaining all important KVs' indexes, these indexes are merged and deduplicated because indexes from different

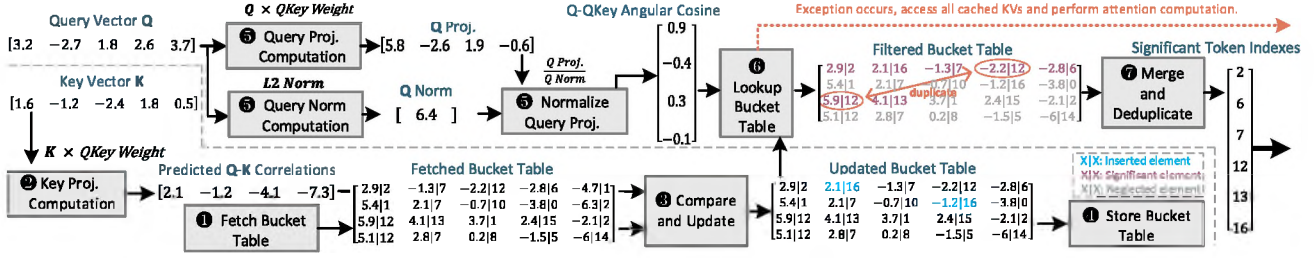


Fig. 4. Workflow using associative arrays for selective KV access. Assume the current token index is 16.

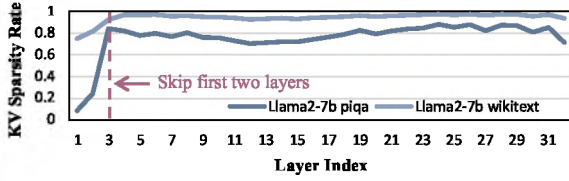


Fig. 5. KV skipping potential of Llama2-7b in Wikitext-2 and PIQA tasks.

buckets may be repeated. Then, a list of KV's indexes to be accessed is obtained for subsequent attention computation.

5) *Further improvement measures*: As shown in Fig.5, we analyze the proportion of attention weights below 0.001 to evaluate the KV sparsity potential of each layer for PIQA and Wikitext-2 in Llama2-7b. It was found that the KV sparsity potential is lower in the initial layers. This situation also exists in other LLMs and tasks. If our proposed technique is applied to the initial layers, it may lead to a drop in accuracy, or require a large table to meet accuracy requirements, resulting in negative gains. Therefore, in this paper, we skip the first 2 layers and only apply the proposed technique to the subsequent layers for all LLMs to achieve high overall gains. Another possible approach is to apply bucket tables of different sizes for different layers, which we leave for future work.

IV. HARDWARE DESIGN

A. Overview

Fig. 6 depicts the overview architecture of our proposed hardware accelerator design. The accelerator consists of two major components: (a) the prediction engine, and (b) the computation engine. The prediction engine implements the proposed algorithm, which updates the bucket table and predicts the KVs required for each attention head. The load/store unit (LSU) then loads these KVs from off-chip memory. Finally, the computation engine performs the attention computation.

B. Prediction Engine

The prediction engine performs projection computation and determines important KVs to be fetched from off-chip memory. It includes the following modules: the processing elements (PEs), a bucket table, and an index merging module. A PE is implemented as a multiply-accumulate (MAC) tree, as shown in Fig. 6(c). The L2-norm required by normalization is calculated by a PE and a square root module. An array of

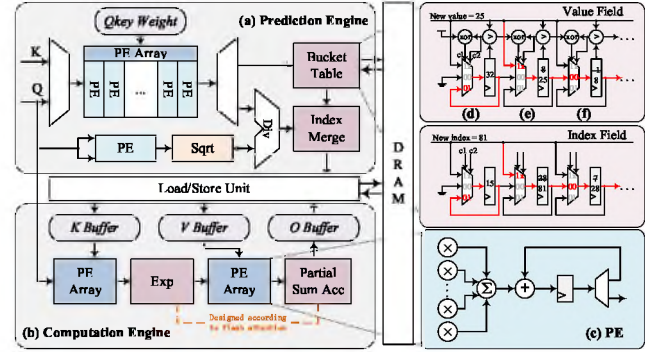


Fig. 6. Overview of Sella accelerator.

stacked PE performs the vector-matrix multiplication operation and is responsible for the computation of predicted $Q \cdot K$ correlation.

Each row of the bucket table has a correlation value field and an index field, and is designed based on shift registers. The value field always maintains a descending order from left to right, this is achieved by a compare-and-insert updating strategy. When a new value-index pair arrives, the new value is first compared with each register, if the new value is smaller, then the register keeps its value (Fig. 6(d)). Otherwise, the registers with smaller values need to be right-shifted (Fig. 6(f)), and the left-most register among these registers (Fig. 6(e)) is updated by the new value. The position of this left-most register is found by the *xor* gates. Therefore, the value field remains ordered after inserting a new value, and the smallest value will be discarded. The index field records the KVs' indexes for the value field, so the control signals (*c1* and *c2*) for multiplexers are the same as their corresponding values. An example of this updating process is presented in Fig. 6, where the activated data paths are highlighted in red.

The index merging module selects and deduplicates the top- N indexes required for computation in each bucket, as mentioned in section III-B3. We adopt a bit-map to achieve the deduplication of indexes. The bit-map consists of M 1-bit registers, where M is the maximum index possible. In this paper, we set M to 2048, which is the maximum context length of the tested networks. When a new index i arrives, we first look up the i -th register in the bit-map, as in Eq. 2, the index is *Valid* when it has not occurred until cycle t . Then, the bit-

map is updated by setting the i -th register to 1, as in Eq. 3, where BM stands for bit-map.

$$Valid_t = !((1 \ll i) \& BM_t) \quad (2)$$

$$BM_{t+1} = (1 \ll i) | BM_t \quad (3)$$

The index merging module's output is serial and is pipelined with LSU. Once a new valid token index is generated, the LSU will fetch the corresponding KVs from off-chip memory.

C. Computation Engine

The computation engine performs the attention calculation. It is designed to be fully pipelined to maximize throughput. The computation engine includes two PE arrays, exponent modules, partial sum accumulators, and on-chip buffers. The two PE arrays are homogeneous to the PE array mentioned in the prediction engine, responsible for $Q \times K^T$ and $Attn \times V$ computation, respectively. The exponent module and partial sum accumulator are designed according to the flash attention algorithm [12], in which the *softmax* operation can be completed in a single traverse on the input, avoiding the stalling problem of *softmax* with a long context. The K and V buffers hold only the selected KVs loaded by LSU which are determined by the prediction engine.

V. EVALUATION

A. Evaluation Methodology

For algorithm-related evaluations, we use the lm-evaluation-harness library [13] and evaluate Sella on three different LLMs: Llama2-7b [2], Opt-6.7b [3] and Pythia-6.9b [14]. Our evaluation benchmarks include PIQA [15], OpenBookQA [16] (OBQA), COPA [17], Winogrande [18] (WG), and Wikitext-2 [19] (Wiki). Specifically, we run PIQA as a 4-shot task, whereas OpenBookQA, COPA, and Winogrande are 16-shot tasks for longer sequences. We report task-specific metrics, e.g, perplexity for Wikitext-2 and accuracy for the rest tasks. We randomly picked 100 samples from Wikitext-2 for clustering to calibrate the QKey weights.

We implemented the hardware in Chisel [20] and synthesized using Synopsis Design Compiler under 22nm CMOS technology to get the chip area and total power at 500MHz working frequency. Table II provides the design parameters and area/power breakdown of Sella. We use HBM2 as the main memory and set the bandwidth to 256 GB/s. The DRAMsim3 [21] is used to evaluate the HBM's access energy. For performance evaluation, we developed a cycle-accurate simulator based on Sanger's open-source codes. During the performance evaluation, we set the first 20% of the sequence as prompts and the remaining 80% as generated tokens for each dataset.

We compare Sella with a server GPU (NVIDIA TITAN RTX), a server CPU (Intel(R) Xeon(R) Gold 5218 CPU @ 2.30GHz), and the state-of-the-art attention accelerators SpAtten [6] and Sanger [7]. For SpAtten, we reproduced the results based on its paper and open-source code. We assumed it first performs 70% sparsity on the prompt KVs, then started token

TABLE II
DESIGN PARAMETERS OF SELLA.

Module	Parameters	Area(mm ²)	Power(mW)
Prediction Engine	8×8 MACs 8×128 bucket table 8KB QKey weight buffer	0.3	537.7
Computation Engine	Two 32×16 MACs 16 Exp. modules 32 partial sum accumulators 256KB KV buffer 16KB output buffer	6.4	1106.5
Total	-	6.7	1644.2

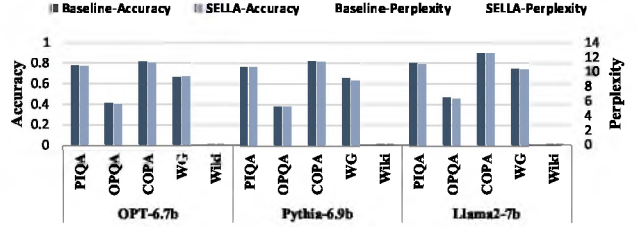


Fig. 7. Accuracy of baseline and Sella.

generation. For Sanger, we obtained its performance from its open-source simulator. For a fair comparison, we scale the bandwidth of SpAtten and Sanger to 256GB/s.

In the following experiments, unless stated otherwise, for Sella, BN is set to 64, α is set to 0.0625, δ is set to 0.6, and this technique is not employed for the first two layers of all the networks.

B. Evaluation Results

1) *Accuracy Results*: We compared Sella's inference accuracy with the baseline (obtained with the prediction engine off). The results are shown in Fig. 7. Sella demonstrates excel-

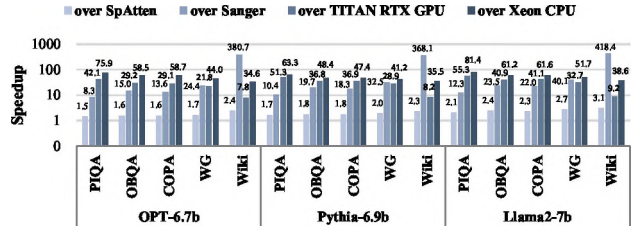


Fig. 8. Speedup over other accelerators.

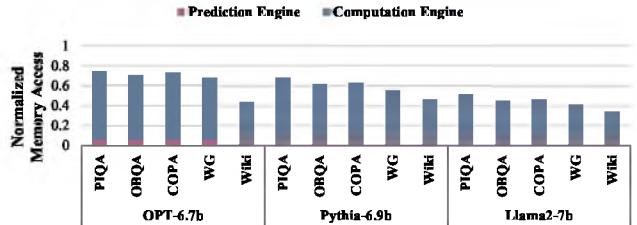


Fig. 9. Normalized memory access breakdown of Sella.

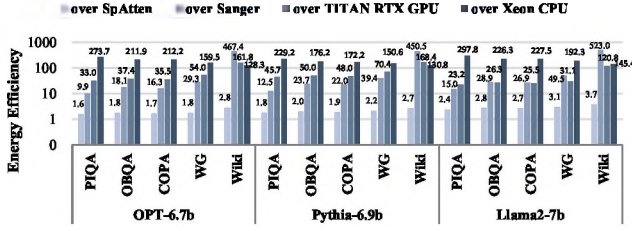


Fig. 10. Energy efficiency over other accelerators.

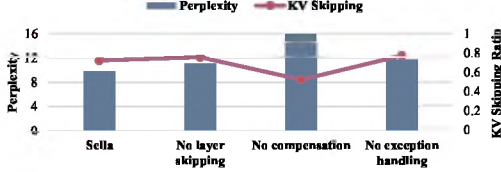


Fig. 11. Comparison of the effects of enabling and disabling layer skipping, accuracy compensation, and exception handling.

lent algorithmic performance. Across four question-answering tasks on three networks, the accuracy drops by 1.4% at most. For the Wikitext-2 dataset, The perplexity increase was less than 7%.

2) *Comparison with other accelerators:* Fig.8 shows the speedup of Sella compared to SpAtten, Sanger, TITAN RTX GPU, and Xeon CPU. Sella demonstrates superior performance over other accelerators across five datasets on three networks. On average, Sella achieves a speedup of $2.1\times$, $93.8\times$, $31.4\times$, and $53.5\times$ over SpAtten, Sanger, TITAN RTX GPU, and Xeon Gold 5218 CPU. The speedup mainly comes from Sella's ability to skip unnecessary KVs during each token generation at a very low cost, reducing substantial off-chip KV cache access and attention computation. Sanger performs poorly on Wikitext-2 because the Wikitext-2 sequence length is long, and Sanger does not support KV cache, requiring KV recomputation with each token generation.

Fig.9 shows the ratio of Sella's memory access breakdown with prediction engine enabled versus disabled. It can be seen that it reduces a large amount of off-chip memory access, reaching up to 66% (Wikitext-2 in Llama2-7b). The rest of the observations from the figure are as follows: (1) the overhead introduced using associative arrays is relatively small, accounting for at most 6.3% of the memory access before optimization. (2) Gains are greater on Wikitext-2. This is due to the long sequence length of the Wikitext-2 task. The speedup improves with longer sequence lengths, as it skips more KV access in these cases.

Fig.10 shows the energy efficiency results. On average, Sella achieves an energy efficiency of $2.3\times$, $115.5\times$, $62.1\times$, and $195.6\times$ over SpAtten, Sanger, TITAN RTX GPU, and Xeon Gold 5218 CPU. The significant reduction in KVs' off-chip access contributes to the superior energy efficiency.

3) *Effectiveness of Layer Skipping, Accuracy Compensation, and Exception Handling:* Fig.11 demonstrates the

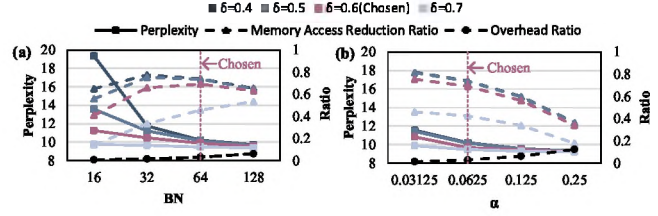


Fig. 12. Design space exploration of BN , α and δ . Overhead means the memory access introduced by the prediction engine.

effectiveness of the several proposed techniques. The KV skipping ratio represents the percentage of KVs skipped by the proposed technique, with higher values indicating better inference performance. Experiments were performed on the Wikitext-2 with the Llama2-7b. The perplexity significantly increased after disabling the layer-skipping and exception-handling techniques. Despite the KV skipping ratio also increasing slightly, the model's accuracy was no longer as expected. When disabling accuracy compensation, there was a sharp increase in perplexity, but the KV skipping ratio decreased. This is because disabling accuracy compensation led to significant inference errors, causing the Q vectors across layers to deviate greatly from cluster centroids, triggering many exception-handling events.

4) *Design Space Exploration:* In this section, we explore the design space of BN , α , and δ in Sella. Fig.12 shows the results of the design space exploration on the Wikitext-2 using the Llama2-7b model. The figure indicates that a larger δ corresponds to lower perplexity, but also a lower memory access reduction ratio. This is because a higher δ implies stricter exception-handling criteria, leading to more exceptions. As BN increases, perplexity decreases, while the reduction ratio rises at δ values of 0.4, 0.5, and 0.6 before slightly decreasing, and steadily rising at $\delta = 0.7$. This is because, at lower δ values, when BN increases to a certain extent, the memory access from accuracy compensation exceeds the reduction from exception handling, leading to a slight decrease in the reduction ratio; the opposite occurs at higher δ values. For α , increasing its value leads to lower perplexity and reduced reduction ratio. This is because a larger α introduces more important KVs. Furthermore, as BN and α increase, the memory access overhead introduced by the associative arrays also steadily grows. By balancing the trade-offs between accuracy and memory access, we selected $BN=64$, $\alpha=0.0625$, and $\delta=0.6$ as the default settings.

VI. CONCLUSION

This paper introduces Sella, a software-hardware co-design solution to speed up LLM token generation by reducing cached KV access and computations via cluster-based associative array. Experiments show that Sella can deliver significant performance gains and reduce substantial memory access without significant accuracy loss.

REFERENCES

- [1] OpenAI, “Gpt-4 technical report,” 2024. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, and Others, “Llama 2: Open foundation and fine-tuned chat models,” 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [3] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, “Opt: Open pre-trained transformer language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2205.01068>
- [4] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, “Large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.06196>
- [5] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh *et al.*, “A3: Accelerating attention mechanisms in neural networks with approximation,” in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 328–341.
- [6] H. Wang, Z. Zhang, and S. Han, “Spatten: Efficient sparse attention architecture with cascade token and head pruning,” in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2021, pp. 97–110.
- [7] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, “Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture,” ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 977–991. [Online]. Available: <https://doi.org/10.1145/3466752.3480125>
- [8] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, “Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 692–705.
- [9] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, “Dota: detect and omit weak attentions for scalable transformer acceleration,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’22. New York, NY, USA: Association for Computing Machinery, 2022, p. 14–26. [Online]. Available: <https://doi.org/10.1145/3503222.3507738>
- [10] Y. Zhao, D. Wu, and J. Wang, “Alisa: Accelerating large language model inference via sparsity-aware kv caching,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 1005–1017.
- [11] A. Vaswani, “Attention is all you need,” *Advances in Neural Information Processing Systems*, 2017.
- [12] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, “Flashattention: Fast and memory-efficient exact attention with io-awareness,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 16 344–16 359, 2022.
- [13] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h *et al.*, “A framework for few-shot language model evaluation, 12 2023,” *URL* <https://zenodo.org/records/10256836>, vol. 7.
- [14] S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff *et al.*, “Pythia: A suite for analyzing large language models across training and scaling,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 2397–2430.
- [15] Y. Bisk, R. Zellers, J. Gao, Y. Choi *et al.*, “Piqa: Reasoning about physical commonsense in natural language,” in *Proceedings of the AAAI conference on artificial intelligence*, vol. 34, no. 05, 2020, pp. 7432–7439.
- [16] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, “Can a suit of armor conduct electricity? a new dataset for open book question answering,” 2018. [Online]. Available: <https://arxiv.org/abs/1809.02789>
- [17] A. Wang, Y. Pruksachatkun, N. Nangia, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, “Superglue: A stickier benchmark for general-purpose language understanding systems,” *Advances in neural information processing systems*, vol. 32, 2019.
- [18] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale,” *Communications of the ACM*, vol. 64, no. 9, pp. 99–106, 2021.
- [19] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” 2016. [Online]. Available: <https://arxiv.org/abs/1609.07843>
- [20] J. Bachrach, H. Vo, B. Richards, Y. Lee, A. Waterman, R. Avižienis, J. Wawrzyniek, and K. Asanović, “Chisel: constructing hardware in a scala embedded language,” in *Proceedings of the 49th Annual Design Automation Conference*, 2012, pp. 1216–1225.
- [21] S. Li, Z. Yang, D. Reddy, A. Srivastava, and B. Jacob, “Dramsim3: A cycle-accurate, thermal-capable dram simulator,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 106–109, 2020.